

Overview

Mobile Build Blueprint - webapp-main

Repository: kashyapempiricinfotech-sys/webapp-main, branch main. Evidence snapshot: 34 files; read-only source analysis; application was not run and no application endpoints were contacted.

Snapshot cutoff: Jul 10, 2026 at 18:00 IST (12:30 UTC). Resolved commit:

368cec4aff9f293a2564eb6c35f3d9045b343c2c, committed at 17:00 IST (11:30 UTC).

Commit selection used commit/landing timestamps, not author timestamps. Two later main-branch commits are deliberately excluded; the post-cutoff experimental scanner is not part of this blueprint.

Evidence convention: every existing-system claim cites repository path and line; recommendations or behavior not directly implemented are marked (inferred). No credential values are reproduced.

Tab directory

- Architecture — web/server stack, active and dormant modules, registered endpoint groups, authentication, and integrations.
- Screen Inventory — all six source-backed screens and four reusable UI building blocks, including exact direct/indirect reuse counts.
- Mobile Stack — cross-platform, bridge, testing, and delivery alternatives with named vendor tiers, current published pricing, and a recommended budget.
- User Flow — implementation-aware screen-to-screen journeys, role/feature branches, checkout return links, and broken transitions.
- API Contract & Gaps — complete server endpoint inventory, authentication boundaries, and client/server mismatches.
- Data Model — buried PostgreSQL/Prisma User, Invoice, and NotificationSetting entities, relationships, and runtime gaps.
- Delivery Plan — four dependency-ordered implementation phases mapped across Sprints 1–6 with build-ready exit criteria.
- Risks & Open Questions — prioritized security, integration, navigation, deployment, and product decisions.
- Secrets found (review) — credential-shaped source locations and remediation guidance; values are intentionally omitted.

Source: [Pinned GitHub commit](#).

Architecture

Architecture

Runtime and technology

- Frontend: Next.js 15, React 19, TypeScript, file-based screens under app/. Source: package.json:5-25; app/login/page.tsx:1-6.
- API runtime: Express 5 with JSON middleware, auth-guarded route groups, and manifest-registered dynamic routes. Source: server/index.ts:1-26; server/routes/manifest.ts:3-7.
- Persistence definition: PostgreSQL datasource and Prisma client generator in a nested package; no active database reads/writes or Prisma runtime dependency are present in this snapshot. Source: packages/data/prisma/schema.prisma:1-32; package.json:11-25; server/routes/invoices.ts:6-9.
- Existing quality tooling: TypeScript typecheck and Vitest commands are declared; no test files or CI workflow files exist among the snapshot's 34 files. Source: package.json:5-9; repository snapshot inventory.

Business modules and execution state

- Authentication/session: login, refresh, Bearer JWT verification, and a separate deprecated but registered cookie-session profile/logout path. Source: server/routes/auth.ts:6-12; server/middleware/requireAuth.ts:4-12; server/routes/legacy.ts:4-7.
- Invoices/payments: invoice list currently returns a stubbed empty array; authenticated invoice checkout creates a live Stripe Checkout session with a fixed minor-unit amount. Source: server/routes/invoices.ts:6-9; src/integrations/stripe.ts:3-10.
- Preferences: authenticated GET/PUT notification preferences are registered; the settings screen instead calls an unregistered /api/settings route. Source: server/routes/notifications.ts:3-5; src/services/settings.ts:3-4.
- Public catalog and protected audit events are registered dynamically from routeManifest; they would be missed by searching only router.get/router.post. Source: server/index.ts:21-23; server/routes/manifest.ts:5-7.
- Push: notification-center screen is disabled unless its feature flag is true; client installation registration targets a route absent from server wiring. Source: app/notifications-center/page.tsx:3-5; src/services/notifications.ts:3-6; server/index.ts:16-23.
- Admin reporting: screen checks user.role on the client-facing page; reportingAggregator is explicitly unwired dead code and must not be described as an active service. Source: app/admin-reports/page.tsx:4-6; src/services/reportingAggregator.ts:3-5.

Endpoint and authentication boundaries

Public: POST /api/auth/login, POST /api/auth/refresh, GET /api/catalog, and the standalone health handler at api/health.ts (mounted public URL inferred). Bearer-protected: GET

/api/invoices; POST /api/invoices/:invoiceId/checkout; GET and PUT /api/notifications/preferences; GET /api/audit-events. Legacy cookie session only: GET /api/legacy/profile and POST /api/legacy/logout. Sources: server/index.ts:16-23; server/routes/auth.ts:6-12; server/routes/invoices.ts:6-9; server/routes/legacy.ts:4-7; server/routes/manifest.ts:5-7; server/routes/notifications.ts:3-5; api/health.ts:1-3.

Authentication flow and implementation constraints

- Login assigns a role based on email suffix and signs a JWT with a static fixture subject; password validation, token expiration, secure refresh rotation, and tenant-aware subject assignment are not implemented. Source: server/routes/auth.ts:6-12.
- Protected routers invoke requireAuth before handlers; middleware extracts Authorization: Bearer and verifies against configured public-key material, returning 401 on missing/invalid tokens. Source: server/index.ts:17-18; server/middleware/requireAuth.ts:4-12.
- Legacy session has httpOnly enabled at this snapshot but does not yet contain the later sameSite hardening; mobile should not inherit cookie auth. Source: server/auth/legacySession.ts:4-7; server/routes/legacy.ts:4-7.
- Mobile design (inferred): central auth client, Keychain/Keystore-backed token storage, refresh single-flight, role-aware navigation, logout revocation, and no secret values in client bundles.

Third-party integrations

- Stripe is active at server startup/use and generates checkout success/cancel URLs under the webapp:// custom scheme; mobile must own and validate deep-link return handling (inferred). Source: src/integrations/stripe.ts:3-10; server/routes/invoices.ts:6-9.
- PostHog initialization is invoked from server startup, while the config example uses a different variable name from the integration; the invoice-opened event function exists but is not invoked by any screen. Source: server/index.ts:2,13; src/integrations/posthog.ts:3-8; .env.example:3.
- Sentry is wired but disabled unless SENTRY_ENABLED is true; DSN belongs in deployment configuration. Source: server/index.ts:3,14; src/integrations/sentry.ts:3-6; .env.example:4.
- Cloud bootstrap contains credential-shaped material in a client-reachable module; classify for security review, not as evidence of a proven live cloud integration. Source: src/config/cloud.ts:1-4.

Screen Inventory

Screen Inventory

Coverage includes every `app/*/page.tsx` present at the pinned commit; routes added after the cutoff are excluded. Screen URL paths are derived from Next.js folder routing (inferred). Reuse counts include barrel-export aliases and higher-order wrappers.

Screen catalog

Screen / route	Purpose and behavior	Access	Source
Login /login	Sign in and start JWT session; current credentials are fixture-only.	Public	app/login/page.tsx:1-6
Dashboard /dashboard	Account snapshot and invoice CTA; navigation handler missing.	Signed in (inferred)	app/dashboard/page.tsx:1-4
Invoices /invoices	Show invoice card and tracked create CTA; currently fixture data.	Signed in (inferred)	app/invoices/page.tsx:1-9
Settings /settings	Save payment-reminder preferences; client route mismatch.	Signed in (inferred)	app/settings/page.tsx:1-6
Notifications /notifications-center	Payment-reminder center; <code>FEATURE_PUSH</code> controls visibility.	Flag-enabled + signed in (inferred)	app/notifications-center/page.tsx:1-5
Admin reports /admin-reports	Restricted revenue report; page-level role gate only.	Admin role	app/admin-reports/page.tsx:1-6

Reusable component catalog

Reusable component	Purpose	Reuse count	Source
AppShell	Shared titled screen frame.	5 screen usages	src/components/AppShell.tsx:3-4
ActionButton / PrimaryCta	Reusable primary action; exported under alias.	4 screens; 1 indirect via HOC	src/components/ActionButton.tsx:3-4; src/components/index.ts:1; app/invoices/page.tsx:5
InvoiceCard	Invoice number/status card.	1 screen usage	src/components/InvoiceCard.tsx:3-4; app/invoices/page.tsx:9

Reusable component	Purpose	Reuse count	Source
withTelemetry	Higher-order wrapper adding analytics component marker.	1 wrapped CTA	src/components/withTelemetry.tsx:3-5; app/invoices/page.tsx:5

Counting and state notes

- AppShell is imported by dashboard, invoices, settings, notifications center, and admin reports: five screen call sites; notifications is feature-gated.
- ActionButton is exported as PrimaryCta and used on login, dashboard, invoices, and settings: four screen call sites, including the invoice screen's indirect withTelemetry(PrimaryCta) wrapper. Sources: src/components/index.ts:1; app/invoices/page.tsx:1-5.
- InvoiceCard is rendered once on the invoice list; withTelemetry is instantiated once for the invoice CTA. Sources: app/invoices/page.tsx:2-9; src/components/InvoiceCard.tsx:3-4; src/components/withTelemetry.tsx:3-5.
- The invoices screen displays a hard-coded sample record rather than calling listInvoices; dashboard CTA has no click handler, and settings uses a literal placeholder session token. Sources: app/invoices/page.tsx:5-9; app/dashboard/page.tsx:4; app/settings/page.tsx:6.

Mobile Stack

Mobile Stack

Published pricing verified on **Aug 4, 2026** . Prices are USD, before tax, and usage assumptions are explicit. Open-source SDK tiers are priced at \$0/month; hosted-service subscriptions are counted once, even when the same Expo plan supports multiple capabilities.

Option comparison

Category	Option and published plan price	Tradeoff / monthly assumption	Decision
Framework	React Native + Expo SDK — Free \$0/month	Existing React/TypeScript/domain sharing; native UI rewrite required.	Recommended
Framework	Flutter SDK — free/open-source \$0/month	Dart rewrite; strong rendering and mature platform ecosystem.	Alternative
Native bridges	Expo Modules / Expo SDK Free \$0/month	Secure store, notifications, deep links, web auth; custom Swift/Kotlin modules.	Recommended
Native bridges	Flutter platform channels / Flutter SDK \$0/month	Dart-to-Swift/Kotlin bridge; more migration and dual-platform glue.	Alternative
Testing	Expo EAS Starter + Maestro: \$19/month + \$0.05/test job	100 jobs ≈ \$24/month plus metered CI minutes; Expo subscription overlaps CI.	Recommended when hosted E2E needed
Testing	Firebase Test Lab Spark \$0; Blaze 20 physical hours ≤ \$100/month	Spark: 10 virtual + 5 physical runs/day; Blaze \$5/physical-hour, \$1/virtual-hour above allowances.	Device-matrix option
CI/CD	Expo EAS Starter \$19/month; Production \$199/month	Starter includes \$45 build credit; Production \$225 and two concurrencies.	Recommended: Starter
CI/CD	Codemagic Individual \$0 for 500 M2 minutes; 700 minutes ≈ \$19/month	First 500 monthly M2 minutes free; additional \$0.095/minute.	Alternative; 1 parallel build

Recommendation and build shape

- Choose React Native + Expo + TypeScript + Expo Router (inferred):
React/TypeScript skills, shared domain types and API service contracts, file-based

route familiarity, secure storage, deep links, and push integration provide the shortest migration from the existing React code. DOM-based components still need rewriting with native primitives. Sources: package.json:14-25; src/types.ts:1-16; src/components/ActionButton.tsx:3-4.

- Bridge package shortlist (inferred): expo-secure-store for Keychain/Keystore session storage; expo-notifications for APNs/FCM; expo-linking and expo-web-browser for Stripe browser/deep-link return; Expo Modules/config plugins for Swift/Kotlin escape hatches. Source evidence for required capabilities: src/integrations/stripe.ts:5-10; src/services/notifications.ts:5-6; server/middleware/requireAuth.ts:4-8.
- Testing shape (inferred): keep existing typecheck/Vitest for shared pure logic, add React Native Testing Library for screen behavior, Maestro for login/invoice/checkout/preferences journeys, and Firebase physical-device smoke coverage. Source: package.json:5-9; src/services/invoices.ts:4-12.
- CI/CD shape (inferred): protected GitHub pull requests run typecheck, unit tests, lint, secret scanning, and API-contract checks; EAS Build creates signed internal Android/iOS artifacts; staged channels, TestFlight/Play internal distribution, rollback, and release approval are added before public publication.

Costed recommendation

Recommended baseline: Expo EAS Starter \$19/month + Firebase Test Lab Spark \$0/month + Apple Developer Program annual membership amortized to \$8.25/month = approximately \$27.25/month recurring; Google Play Console registration adds \$25 once. A 100-job monthly Maestro example adds approximately \$5 plus any metered CI minutes, for approximately \$32.25/month. Replace Spark with 20 chargeable physical-device hours on Firebase Blaze for up to approximately \$100/month extra; included daily allowances may reduce the actual total.

Vendor sources: [Expo pricing](#); [Expo Modules API](#); [Flutter SDK FAQ](#); [Flutter platform channels](#); [Firebase Test Lab pricing](#); [Codemagic pricing](#); [Apple Developer enrollment](#); [Google Play registration](#).

User Flow

User Flow

The sequence below distinguishes code-proven API interactions from navigation transitions that the current web screens do not wire. Any arrow representing desired navigation, secure storage, or screen composition is marked (inferred).

Primary signed-in journey

- 1. Launch -> Login screen (inferred). The login view renders a primary CTA that calls `signIn`. Source: `app/login/page.tsx:1-6`.
- 2. Login CTA -> `POST /api/auth/login` -> receive JWT and `SessionUser`. Store token in platform-secure storage and derive role-aware navigation (inferred). Sources: `src/services/auth.ts:3-6`; `server/routes/auth.ts:6-9`.
- 3. Authenticated landing -> Dashboard (inferred). Render account overview; dashboard's Review invoices CTA requires a navigation handler because the existing JSX has none. Source: `app/dashboard/page.tsx:3-4`.
- 4. Dashboard -> Invoices list (inferred) -> `GET /api/invoices` with Bearer auth. Replace the existing hard-coded fixture card and implement loading, empty, error, and retry states. Sources: `app/invoices/page.tsx:5-9`; `src/services/invoices.ts:4-7`; `server/routes/invoices.ts:6`.
- 5. Invoice card -> Invoice detail/checkout confirmation (new screen, inferred) -> `POST /api/invoices/:invoiceId/checkout` -> Stripe-hosted browser session (inferred). Sources: `src/services/invoices.ts:10-12`; `server/routes/invoices.ts:7-9`; `src/integrations/stripe.ts:5-10`.
- 6. Stripe return -> `webapp://invoices/{invoiceId}?checkout=success` or `checkout=cancelled` -> validate invoice ownership, refresh invoice state, and return to invoice details (inferred). Source: `src/integrations/stripe.ts:8-10`.
- 7. Dashboard -> Settings (inferred) -> load/save preferences through `GET/PUT /api/notifications/preferences`. Remap the current settings client away from unregistered `PUT /api/settings`. Sources: `app/settings/page.tsx:3-6`; `src/services/settings.ts:3-4`; `server/routes/notifications.ts:3-5`.

Conditional branches

- Push-enabled branch (inferred): Dashboard -> Notifications center when `FEATURE_PUSH` is enabled -> request platform permission -> register installation after implementing the currently missing `POST /api/notifications/installations` contract. Sources: `app/notifications-center/page.tsx:3-5`; `src/services/notifications.ts:3-6`; `server/index.ts:16-23`.
- Admin branch: authenticated user with admin role -> Admin reports (navigation inferred); preserve server-side authorization rather than relying on the current page-only role check. Sources: `app/admin-reports/page.tsx:4-6`; `server/routes/manifest.ts:5-7`.
- Session failure branch (inferred): any protected call receives 401 -> one refresh attempt -> retry once or clear secure storage and return to Login. Existing refresh endpoint is public and returns static fixture data, so production refresh semantics

must be implemented first. Sources: `server/middleware/requireAuth.ts:4-11;`
`server/routes/auth.ts:12.`

Implementation gate

No scanner, scanner route, or experimental scanner screen appears in the pinned snapshot; those belong to a later commit and must not enter the initial mobile scope.

API Contract & Gaps

API Contract & Gaps

Routes below are derived from server/index.ts mounts, Express router definitions, dynamic routeManifest registration, and the standalone api/health.ts handler. The health URL is inferred from the source layout; all others are explicitly registered.

Registered endpoint inventory

Method	Path	Auth / implementation state	Source
GET	/api/health	Public (URL inferred)	api/health.ts:1-3
POST	/api/auth/login	Public	server/index.ts:16; server/routes/auth.ts:6-9
POST	/api/auth/refresh	Public; unsafe static response	server/index.ts:16; server/routes/auth.ts:12
GET	/api/invoices	Bearer JWT	server/index.ts:17; server/routes/invoices.ts:6
POST	/api/invoices/:invoiceId/checkout	Bearer JWT	server/index.ts:17; server/routes/invoices.ts:7-9
GET	/api/notifications/preferences	Bearer JWT	server/index.ts:18; server/routes/notifications.ts:4
PUT	/api/notifications/preferences	Bearer JWT	server/index.ts:18; server/routes/notifications.ts:5
GET	/api/legacy/profile	Legacy cookie session	server/index.ts:19; server/routes/legacy.ts:5-6
POST	/api/legacy/logout	Legacy cookie session	server/index.ts:19; server/routes/legacy.ts:5-7
GET	/api/catalog	Public; dynamic manifest	server/index.ts:21-23; server/routes/manifest.ts:6
GET	/api/audit-events	Bearer JWT; dynamic manifest	server/index.ts:21-23; server/routes/manifest.ts:7
PUT	/api/settings	NOT REGISTERED; called by client	src/services/settings.ts:3-4; server/index.ts:16-23
POST	/api/notifications/installations	NOT REGISTERED; called by client	src/services/notifications.ts:5-6; server/routes/notifications.ts:3-5

Client/server contract defects

- P0 — Settings client sends PUT /api/settings, but no server router registers that path. Retarget the mobile/web client to PUT /api/notifications/preferences or add an explicitly documented settings adapter. Sources: src/services/settings.ts:3-4; server/routes/notifications.ts:3-5; server/index.ts:16-23.
- P0 — Push-registration client sends POST /api/notifications/installations, but notificationRouter exposes only GET/PUT /preferences. Implement authenticated installation create/update/revoke contracts before enabling push. Sources: src/services/notifications.ts:3-6; server/routes/notifications.ts:3-5.
- P0 — Login request serializes JSON without a Content-Type header; Express only parses application/json by default, so request.body may not match handler expectations (inferred). Sources: src/services/auth.ts:3-5; server/index.ts:11-12; server/routes/auth.ts:6-9.
- P0 — JWT signer and verifier use separate configured key inputs with insecure literal fallbacks; algorithm, expiry, issuer, audience, key rotation, and refresh protection are not defined. Sources: server/routes/auth.ts:6-12; server/middleware/requireAuth.ts:4-11.
- P1 — Invoice list and preferences are stubs; checkout does not verify ownership or compute amount/currency from persisted Invoice. Sources: server/routes/invoices.ts:6-9; src/integrations/stripe.ts:5-10; packages/data/prisma/schema.prisma:18-25.
- P1 — PostHog's configured public variable name and .env.example's documented variable name differ; telemetry initialization may run without a usable project key (inferred). Sources: src/integrations/posthog.ts:3-4; .env.example:3.
- P1 — Admin report authorization exists in the screen, not in a dedicated server-side admin route or role guard; audit events only require general authentication. Sources: app/admin-reports/page.tsx:4-6; server/routes/manifest.ts:5-7.

Data Model

Data Model

Authoritative schema is nested under `packages/data/prisma/schema.prisma`, not the repository root. It declares a PostgreSQL datasource and a generated Prisma client, but the active server currently uses response stubs rather than the schema.

Entities and relationships

Entity	Fields	Relationships	Source
User	id PK; email unique; role String	1-to-many Invoice; optional 1-to-1 NotificationSetting	<code>packages/data/prisma/schema.prisma:10-15</code>
Invoice	id PK; ownerId FK; number; totalMinor; currency; status	belongs to User through ownerId	<code>packages/data/prisma/schema.prisma:18-25</code>
NotificationSetting	userId PK/FK; paymentReminders default true	belongs to exactly one User	<code>packages/data/prisma/schema.prisma:28-31</code>

Shared TypeScript contract

- SessionUser: id, email, role; Role union supports user and admin. Source: `src/types.ts:1-7`.
- Invoice: id, ownerId, number, totalMinor, currency union USD/INR, and status union draft/open/paid. Source: `src/types.ts:9-16`.

Migration and ownership requirements

- Enforce User 1-to-many Invoice ownership and User 1-to-optional-1 NotificationSetting on every protected query (inferred). Source: `packages/data/prisma/schema.prisma:10-31`.
- Reconcile Prisma's unvalidated String role/currency/status fields with stricter TypeScript unions through database enums or validation (inferred). Sources: `packages/data/prisma/schema.prisma:10-30`; `src/types.ts:1-16`.
- Implement a real repository/Prisma client, migrations, deterministic fixtures, pagination, and transaction-safe checkout price lookup before relying on mobile persistence (inferred). Current `package.json` contains no Prisma runtime/tooling dependency. Sources: `package.json:11-25`; `server/routes/invoices.ts:6-9`.
- Define push-installation, refresh-session, and audit-event tables before feature enablement; these entities are not present in the pinned schema (inferred). Sources: `packages/data/prisma/schema.prisma:10-31`; `src/services/notifications.ts:5-6`; `server/routes/auth.ts:12`.

Delivery Plan

Delivery Plan

Draft delivery plan: no deployment or release action is authorized. Foundational work precedes UI feature work, and every phase issue remains in Jira To Do.

Phase-to-sprint schedule

Phase issue summary	Sprint span	Scope	Definition of done
Foundation and auth	Sprints 1–2	Repo/bootstrap, Expo setup, secure JWT, route contracts, role controls, secrets, CI skeleton.	Authenticated device build; 401/refresh tests; credential remediation plan.
Core screens and checkout	Sprints 2–4	Login, dashboard, invoices/details, settings, admin role gating, Stripe return links.	All core journeys and loading/error states work against agreed contracts.
Notifications and analytics	Sprints 4–5	Preference contracts, push installation schema/API, feature gating, PostHog, Sentry policy.	Opt-in/opt-out and telemetry tests pass; disabled mode stays safe.
Testing and release readiness	Sprints 5–6	Unit/integration/Maestro/device tests, security/accessibility, signed internal builds, store readiness, rollback.	Internal iOS/Android release candidates and review checklist pass; no public release.

Critical dependency order

- Foundation and auth must finish secure token handling, role-aware authorization, API base/config separation, contract fixes, secret remediation plan, and signed build scaffolding before protected screens or payment journeys are considered complete.
- Core screens depend on Foundation; implement Login, Dashboard, Invoices, invoice-detail/checkout return, Settings, and role-gated Admin Reports with explicit loading/empty/error/401 states.
- Notifications and analytics depend on both Foundation and contract/schema fixes; remain feature-disabled until installation endpoints, consent, APNs/FCM, analytics-key alignment, and Sentry policy are verified.
- Testing and release spans the later sprints but begins test harness setup earlier (inferred); finish end-to-end device coverage, accessibility, performance, crash monitoring, store metadata, signed internal candidates, and rollback readiness without publishing.

Global definition of done

All accepted mobile flows map to a code-backed or explicitly recommended contract; no credential values appear in artifacts; JWT/cookie coexistence and admin access are tested; checkout return links are validated; push remains gated; CI, device tests, release checklist, and security review pass; Jira issues stay To Do.

Risks & Open Questions

Risks & Open Questions

Priority meanings: P0 blocks trustworthy mobile implementation; P1 requires an explicit product/security decision before release; P2 improves delivery quality.

Risk register

Priority	Risk	Mitigation / open decision	Evidence
P0	Hard-coded credential-shaped material and fallback signing values.	Secrets tab; security owner rotates/revokes and enables merge-blocking scans.	src/config/cloud.ts:2; src/services/notifications.ts:3; server/routes/auth.ts:8
P0	Unauthenticated/static refresh and email-derived roles; no token lifetime policy.	Build managed JWT/refresh rotation, server-side role guards and threat-model tests.	server/routes/auth.ts:6-12; server/middleware/requireAuth.ts:4-11
P0	Missing settings and push installation contracts.	Resolve contract before wiring UI or enabling push.	src/services/settings.ts:3-4; src/services/notifications.ts:5-6
P1	Stubbed persistence, empty invoice API, no ownership checks, hardcoded checkout amount.	Implement Prisma repositories and tenant-scoped checkout.	server/routes/invoices.ts:6-9; packages/data/prisma/schema.prisma:10-31
P1	Stripe custom-scheme callbacks can be spoofed or mismatched.	Register universal/app links or validate signed return/session state (inferred).	src/integrations/stripe.ts:5-10
P1	Feature-gated notification screen and disabled Sentry may be mistaken for production-ready capabilities.	Require explicit feature policy, APNs/FCM setup and error-reporting consent.	app/notifications-center/page.tsx:3-5; src/integrations/sentry.ts:3-6
P1	PostHog environment-variable mismatch and unused event emitter.	Align deployment config and add event schema/consent tests.	src/integrations/posthog.ts:3-8; .env.example:3
P2	No checked-in tests, lockfile or CI configuration at pinned commit.	Add reproducible dependency lock, test fixtures and protected pipelines.	package.json:5-9; snapshot inventory

Product and platform decisions

- Should admin reporting be in the first mobile release, and which backend role/tenant authorization policy is authoritative? Sources: `app/admin-reports/page.tsx:4-6`; `src/types.ts:1-7`.
- Is the intended payment experience hosted Stripe Checkout, PaymentSheet, or another native flow? Current code only proves server-created Checkout URLs and custom deep-link returns. Source: `src/integrations/stripe.ts:5-10`.
- Which invoice statuses/currencies are supported, and should fixed amount/currency placeholders be replaced from persisted Invoice? Sources: `src/types.ts:9-16`; `server/routes/invoices.ts:7-9`; `src/integrations/stripe.ts:8`.
- Are notifications required for launch or intentionally deferred? `FEATURE_PUSH` defaults false and the installation registration endpoint is absent. Sources: `.env.example:5`; `app/notifications-center/page.tsx:3-5`; `server/routes/notifications.ts:3-5`.
- Who owns Apple/Google developer enrollment, signing, privacy nutrition labels, APNs/FCM credentials, and staged-release approval? (inferred).
- Will users ever require legacy cookie sessions on mobile, or may those endpoints be deprecated after web-client migration? Sources: `server/routes/legacy.ts:4-7`; `server/auth/legacySession.ts:4-7`.

Secrets found (review)

Secrets found (review)

Values are intentionally omitted. These findings describe only repository locations and the kind of credential-shaped material observed; treat all findings as sensitive until ownership and exposure are reviewed.

Review findings

- HIGH — `src/config/cloud.ts:2` — cloud access-key-shaped identifier embedded in a client-reachable configuration object. Remove from source/client bundles, rotate or verify inactivity, and use short-lived server-side credentials.
- HIGH — `src/services/notifications.ts:3` — long opaque bootstrap/proof string embedded in a client service and later included in registration payload construction. Revoke/replace, remove from bundles/history where appropriate, and mint short-lived server-issued proofs.
- HIGH — `server/routes/auth.ts:8` — literal fallback material for JWT signing. Remove fallback, require managed secret/key configuration, rotate signing material, and define algorithm/issuer/audience/expiry.
- HIGH — `server/auth/legacySession.ts:6` — literal fallback material for cookie-session signing. Remove fallback, rotate signing material, set explicit secure/sameSite policy, and plan legacy-session retirement.
- MEDIUM — `server/middleware/requireAuth.ts:8` — literal fallback JWT verification material. Remove fallback and align verification keys with the managed signing/rotation policy.
- MEDIUM — `server/routes/auth.ts:12` — literal token-shaped refresh response. Replace with authenticated, rotating, expiring refresh sessions; never embed token material in source.
- MEDIUM — `app/login/page.tsx:6` — embedded sample sign-in credentials in a shipped UI path. Remove fixture credentials from production bundles and use isolated test fixtures.
- MEDIUM — `app/settings/page.tsx:6` — embedded placeholder session credential material in a client action. Bind requests to the authenticated secure session instead.
- REVIEW — `.env.example:2-3` — credential-shaped example placeholders for third-party services. Verify they are nonfunctional placeholders, preserve variable names only, and scan repository history.

Required controls

Block merge on secret scanning; inventory affected providers; rotate or revoke exposed material; remove fallback literals; move configuration into managed server-side secrets; audit repository history and client bundles; document remediation without copying any original value.